SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

*I **Jehisto rijin(192511022)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Analytical Problem Solving

Problem 1

Water Jug Problem (4-gallon and 3-gallon jugs)

Problem Statement: We need exactly 2 gallons of water in the 4-gallon jug using only a pump, a 4-gallon jug, and a 3-gallon jug.

Explicit Assumptions:

- Jugs can be filled from the pump.
- Water can be poured onto the ground.
- Water can be transferred between jugs.
- No other measuring devices are available.

Steps to Solution:

1. Fill the 3-gallon jug completely.
2. Pour all water from the 3-gallon jug into the 4-gallon jug.
3. Fill the 3-gallon jug again.
4. Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full.
    - At this point, 2 gallons remain in the 3-gallon jug.
5. Empty the 4-gallon jug.
6. Transfer the 2 gallons from the 3-gallon jug into the 4-gallon jug.

Final Result: 4-gallon jug contains **exactly 2 gallons**.

Problem 2

Mars Rover Agent Analysis

i. **Percepts:**

- Camera images, soil composition data, temperature, radiation, dust levels, chemical sensor readings.

ii. **Operating Environment:**

- Partially observable, dynamic, continuous, uncertain, hazardous terrain.

iii. **Actions:**

- Move forward/backward, turn, drill, collect samples, analyze samples, transmit data to Earth.

iv. **Performance Evaluation:**

- Success in collecting and analyzing samples.
- Accuracy of transmitted data.
- Energy efficiency.
- Ability to avoid hazards and continue functioning.

v. **Suitable Agent Architecture:**

- **Hybrid architecture (deliberative + reactive).**
    - Reactive: obstacle avoidance, hazard response.
    - Deliberative: path planning, scientific experiments.
- This ensures adaptability in unpredictable Mars conditions.

Problem 3

8-Queens Problem Formulation

Problem Statement: Place 8 queens on a chessboard such that none attack each other.

Problem Components:

- **State Space:** Each state = configuration of queens on the board.
- **Initial State:** Empty chessboard.
- **Operators:** Place a queen in a row/column ensuring no conflict.
- **Goal State:** 8 queens placed with no two attacking (no same row, column, or diagonal).
- **Search Strategies:**
    - Depth-first search (DFS).
    - Backtracking (undo placements if conflict arises).
    - Heuristic search (minimum conflicts heuristic).

Note: There are 92 distinct valid solutions.

Problem 4 -OLA Cab Booking Agent

Problem Statement: A customer wants to book a cab (mini, micro, sedan, shared, prime, etc.) using the OLA mobile app to travel from source to destination.

Type of Agent:

- **Goal-based problem-solving agent.**
- Chooses cab type and confirms booking to achieve the goal of reaching the destination.

Pseudocode:

```
function OLA_CAB_BOOKING(source, destination, cab_type):
available_cabs = GET_AVAILABLE_CABS(source, cab_type)
```

if available_cabs is empty:

return "No cabs available"

else:

best_cab = SELECT_CAB(available_cabs, destination)

CONFIRM_BOOKING(best_cab, source, destination)

return "Cab booked successfully"

Problem 5 – Uniform Cost Search (UCS)

Objective: Find the least-cost path from the starting warehouse **S** to the destination warehouse **G** using the **Uniform Cost Search (UCS)** algorithm.

Step-by-Step Execution

Initialization:

- Add the start node **S** to the priority queue with cost = 0.
- Queue: [(0, S)]

Step 1:

- Remove **S** (cost 0).
- Expand **S** → neighbors: A, C, G.
- Path costs:
  - S → A = 1
  - S → C = 1
  - S → G = 12
- Queue: [(1, A), (1, C), (12, G)]

Step 2:

- Remove **A** (cost 1).
- Expand **A** → neighbor: B.
- Path cost:
  - S → A → B = 4
- Queue: [(1, C), (4, B), (12, G)]

Step 3:

- Remove **C** (cost 1).
- Expand **C** → neighbor: D.
- Path cost:

    - S → C → D = 4
- Queue: [(4, B), (4, D), (12, G)]

Step 4:

- Remove **B** (cost 4).
- Expand **B** → neighbor: D.
- Path cost:
    - S → A → B → D = 7
- Since 7 > existing cost to D (4), skip.
- Queue: [(4, D), (12, G)]

Step 5:

- Remove **D** (cost 4).
- Expand **D** → neighbor: G.
- Path cost:
    - S → C → D → G = 6
- Update G since 6 < 12.
- Queue: [(6, G), (12, G)]

Step 6:

- Remove **G** (cost 6).
- Goal node reached → Stop search.

Final Results

- **Least-Cost Path:** S → C → D → G
- **Minimum Travel Cost: 6**